

NYC Taxi Analytics Pipeline

Pandas Data Transformation and Relational Logic - Student Assignment

Scenario

You are a data engineer for a mobility analytics team. Build a Pandas pipeline to analyze taxi demand, revenue, pickup hotspots, and borough-level patterns using real NYC Taxi and Limousine Commission trip data.

Dataset

Use the public NYC TLC Trip Record dataset and Taxi Zone Lookup. The TLC page explains that taxi trip records include pickup/dropoff times and locations, trip distances, fares, rate types, payment types, and passenger counts. The trip files are published monthly in Parquet format.

Resource	URL
Official page	https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
Yellow Jan 2024	https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-01.parquet
Yellow Feb 2024	https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-02.parquet
Taxi zones	https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv

Colab note: Start with January only. If memory allows, load January and February and concatenate them. Use selected columns only.

Starter Code

```
import pandas as pd
trip_cols = ["tpep_pickup_datetime", "tpep_dropoff_datetime", "PULocationID",
            "DOLocationID", "passenger_count", "trip_distance",
            "fare_amount", "total_amount", "payment_type"]
jan = pd.read_parquet(jan_url, columns=trip_cols)
feb = pd.read_parquet(feb_url, columns=trip_cols)      # optional
zones = pd.read_csv(zone_url)
trips = pd.concat([jan, feb], ignore_index=True)      # or use jan only
```

Concepts You Must Apply

- Groupby and aggregation: split-apply-combine, count, sum, mean, min, max.
- Joins: left join taxi zones twice - once for pickup and once for dropoff.
- Concat: vertically combine monthly trip files.
- Pivot table: create a wide summary report by hour and borough.
- Melt: convert a wide pivot table into long dashboard-ready format.

Required Pipeline Steps

1. Load and inspect: head(), info(), describe(), shape, dtypes, missing values.
2. Clean invalid trips: remove total_amount <= 0, trip_distance <= 0, and trip_minutes <= 0.
3. Create features: pickup_month, pickup_date, pickup_hour, trip_minutes.
4. Join pickup zone: PULocationID -> LocationID to add pickup_borough and pickup_zone.
5. Join dropoff zone: DOLocationID -> LocationID to add dropoff_borough and dropoff_zone.
6. Group by pickup_borough and pickup_zone to calculate trip_count, total_revenue, avg_revenue, avg_distance, min_distance, max_distance.
7. Group by pickup_month and payment_type to calculate count, total revenue, average fare, min fare, max fare.
8. Create pivot: index=pickup_hour, columns=pickup_borough, values=total_amount, aggfunc=sum.
9. Melt the pivot into columns: pickup_hour, pickup_borough, revenue.

Core Patterns Expected

Left join pattern

```
pickup_zones = zones.rename(columns={"LocationID": "PULocationID",
                                     "Borough": "pickup_borough",
                                     "Zone": "pickup_zone"})
trips = pd.merge(trips, pickup_zones[["PULocationID", "pickup_borough", "pickup_zone"]],
                 on="PULocationID", how="left")
```

Groupby pattern

```
summary = trips.groupby(["pickup_borough", "pickup_zone"]).agg(
    trip_count=("total_amount", "count"),
    total_revenue=("total_amount", "sum"),
    avg_revenue=("total_amount", "mean"),
    avg_distance=("trip_distance", "mean")
).reset_index()
```

Pivot and melt pattern

```
pivot = pd.pivot_table(trips, values="total_amount",
                       index="pickup_hour", columns="pickup_borough",
                       aggfunc="sum", fill_value=0)
long_report = pivot.reset_index().melt(id_vars="pickup_hour",
                                       var_name="pickup_borough",
                                       value_name="revenue")
```

Required Outputs

- pickup_zone_summary.csv - top pickup zones with revenue and trip metrics.
- borough_hour_pivot.csv - wide hourly revenue report by pickup borough.
- borough_hour_long.csv - melted long-format dashboard table.
- payment_month_summary.csv - payment-type aggregation by month.
- join_quality_report - raw rows, cleaned rows, unmatched pickup zones, unmatched dropoff zones, final rows.

Business Questions

- Which pickup zone generated the highest total revenue?
- Which pickup borough is busiest by trip count?
- Which pickup hour has the highest revenue in Manhattan?
- What are the top 10 pickup -> dropoff zone routes by trip count?
- If using two months, did revenue increase or decrease from January to February?

Evaluation Criteria

Area	Marks	Criteria
Loading and inspection	15	Reads data and inspects schema, nulls, dtypes.
Cleaning and features	15	Creates date/hour/month/trip_minutes and removes invalid trips.
Joins	20	Correct pickup/dropoff left joins and join quality checks.
Groupby	20	Multiple aggregations with useful metrics.
Pivot and melt	15	Wide pivot and long dashboard-ready output.
Insights	15	Exports files and explains 3 clear business insights.

Submission Checklist

- Colab notebook or .ipynb file
- All required CSV outputs
- join_quality_report printed
- 3 business insights with evidence from outputs